VirtualReactor

User and Developer Manual

September 2, 2026

Contents

1 Introduction

1.1 Overview

VirtualReactor is a modular Python framework for the mechanistic simulation of chemical reactors. Reaction kinetics, reactor models, thermophysical properties, and numerical integration are implemented as independent components that can be combined within a common simulation framework.

The current implementation supports batch and plug-flow reactors, reversible reaction networks, non-isothermal operation, and HDF5-based storage of simulation results.

1.2 Motivation

Chemical reactor models couple reaction kinetics with reactor-specific mass and energy balances. Implementing both within a single model limits the reusability of reaction mechanisms and complicates systematic variation of reactor configurations and operating conditions.

VirtualReactor separates the chemical reaction model from the reactor description. This enables the same reaction mechanism to be evaluated in different reactor configurations and provides a common basis for systematic parameter studies and optimization.

1.3 Design Philosophy

The framework follows a modular object-oriented design. A `Mechanism` describes reaction kinetics and thermodynamics, while reactor-specific behavior and thermophysical properties are represented by independent models. The `Simulation` class combines these components and performs the numerical integration.

Simulation output is represented independently by `SimulationResult`, allowing numerical results to be analyzed and stored without dependence on the simulation procedure.

1.4 Scope

The current version focuses on deterministic simulations of homogeneous chemical reaction networks in batch and steady-state plug-flow reactors. Temperature-dependent kinetics, reversible reactions, reaction heat, and external heat transfer are supported.

Detailed fluid dynamics, multiphase systems, and spatially resolved transport are outside the current scope.

Future development will extend the framework toward systematic data generation and data-driven reactor optimization, including Bayesian optimization of operating conditions and reactor parameters.

2 Mathematical Framework

$$\boxed{\frac{\mathrm{d}\mathbf{y}}{\mathrm{d}\xi} = \mathcal{T}_{\text{reactor}}\left(\mathbf{s}_{\text{chemical}}\right) + \mathbf{s}_{\text{reactor}}} \tag{2.1}$$

where

$\mathbf{y}$ Reactor state vector. Its definition depends on the reactor model.

For a batch reactor, the state vector is

$$\mathbf{y}_{\text{Batch}} = \begin{pmatrix} \mathbf{c} \\ T \\ p \end{pmatrix}, \qquad \mathbf{c} = \begin{pmatrix} c_1 & c_2 & \cdots & c_{N_s} \end{pmatrix}^{\mathrm{T}}. \tag{2.2}$$

Here, $\mathbf{c}$ denotes the concentration vector, T the reactor temperature, and p the pressure.

For a plug-flow reactor (PFR), the state vector is

$$\mathbf{y}_{\text{PFR}} = \begin{pmatrix} \mathbf{F} \\ T \\ p \end{pmatrix}, \qquad \mathbf{F} = \begin{pmatrix} F_1 & F_2 & \cdots & F_{N_s} \end{pmatrix}^{\mathrm{T}}. \tag{2.3}$$

Here, $\mathbf{F}$ denotes the vector of molar flow rates.

ξ Independent variable with respect to which the reactor state is integrated. For a batch reactor, $\xi = t$, where t is the reaction time. For a PFR, $\xi = V$, where V is the reactor volume.

$\mathbf{s}_{\text{chemical}}$ Chemical source vector calculated by the reaction mechanism. For both batch reactors and PFRs, the species contribution is obtained from the reaction rates and stoichiometry as $\boldsymbol{\nu}\mathbf{r}$. Additional chemical contributions, such as the heat released or consumed by the reactions, may also be included.

$\mathcal{T}_{\text{reactor}}$ Reactor-specific transformation of the chemical source vector. For a constant-volume batch reactor, the species transformation is the identity transformation, such that

$$\frac{\mathrm{d}\mathbf{c}}{\mathrm{d}t} = \boldsymbol{\nu}\mathbf{r}. \tag{2.4}$$

For a PFR, the transformation converts the volumetric chemical source into the derivative of molar flow with respect to reactor volume, such that

$$\frac{\mathrm{d}\mathbf{F}}{\mathrm{d}V} = \boldsymbol{\nu}\mathbf{r}. \tag{2.5}$$

$\mathbf{s}_{\text{reactor}}$ Reactor-specific contribution that is independent of the intrinsic chemical source term. For an ideal adiabatic batch reactor, this contribution may be zero. For a non-isothermal PFR, it may contain contributions such as heat transfer between the reactor and an external jacket.

3 Software Architecture

3.1 Architectural Overview

VirtualReactor follows a modular architecture in which chemical kinetics, reactor physics, thermophysical properties, numerical integration, and data management are represented by separate software components.

The architecture reflects the mathematical formulation introduced in the previous chapter,

$$\frac{\mathrm{d}\mathbf{y}}{\mathrm{d}\xi} = \mathcal{T}_{\text{reactor}}\left(\mathbf{s}_{\text{chemical}}\right) + \mathbf{s}_{\text{reactor}}. \tag{3.1}$$

Each term of this equation can be associated with a specific component of the software architecture. This separation allows reaction mechanisms, reactor models, and physical property models to be developed and exchanged independently.

3.2 Core Components and Initialization

3.2.1 Mechanism

The `Mechanism` class defines the chemical reaction network and its kinetic and thermodynamic parameters. The following reaction network is used as an example throughout this section.

Reaction Network

Consider a system consisting of four chemical species A, B, C, and D connected by three reversible reactions:

$$A \rightleftharpoons B \qquad \text{Reaction 1,} \tag{3.2}$$
$$B \rightleftharpoons C \qquad \text{Reaction 2,} \tag{3.3}$$
$$2A \rightleftharpoons D \qquad \text{Reaction 3.} \tag{3.4}$$

The corresponding stoichiometric matrix is

$$\boldsymbol{\nu} = \begin{bmatrix} -1 & 0 & -2 \\ 1 & -1 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \tag{3.5}$$

where each row represents a chemical species and each column represents one reaction. Negative coefficients correspond to reactants and positive coefficients to products.

Kinetic and Thermodynamic Description

The rate of a chemical reaction is determined by the free-energy barrier between the reactants and the transition state. Within transition state theory, this barrier is expressed as

$$\Delta G^{\ddagger}(T) = \Delta H^{\ddagger} - T\Delta S^{\ddagger}, \tag{3.6}$$

where $\Delta H^{\ddagger}$ and $\Delta S^{\ddagger}$ are the activation enthalpy and activation entropy, respectively.

The corresponding rate constant is given by the Eyring equation,

$$k(T) = \frac{k_{\mathrm{B}}T}{h} \exp\left(\frac{\Delta S^{\ddagger}}{R}\right) \exp\left(-\frac{\Delta H^{\ddagger}}{RT}\right), \tag{3.7}$$

with the Boltzmann constant k_{B}, the Planck constant h, and the universal gas constant R.

The thermodynamic driving force of a reversible reaction is described separately by the reaction Gibbs energy,

$$\Delta G_{\mathrm{r}}(T) = \Delta H_{\mathrm{r}} - T\Delta S_{\mathrm{r}}, \tag{3.8}$$

which determines the equilibrium constant according to

$$K(T) = \exp\left(-\frac{\Delta G_{\mathrm{r}}(T)}{RT}\right). \tag{3.9}$$

Thus, the activation parameters determine how fast a reaction approaches equilibrium, whereas the reaction thermodynamics determine the position of that equilibrium.

Initialization

The reaction network can be initialized in VirtualReactor using the `Mechanism` class:

```
import virtualreactor as vr


mechanism = vr.Mechanism(
    species=['A', 'B', 'C', 'D'],

    stoichiometry=np.array([
        # r1, r2, r3
        [-1,  0, -2],  # Species a
        [ 1, -1,  0],  # Species b
        [ 0,  1,  0],  # Species c
        [ 0,  0,  1]   # Species d
    ]),

    pre_exponential_factors=np.array([
        1.0e7,  # A1
        5.0e7,  # A2
        1.0e6   # A3
    ]),

    activation_energies=np.array([
        50_000.0,  # E1
        60_000.0,  # E2
        45_000.0   # E3
    ]) * ureg.joule / ureg.mol,

    reaction_enthalpies=np.array([
        -40_000.0,  # dH1: A -> B
        -30_000.0,  # dH2: B -> C
        -50_000.0   # dH3: 2 A -> D
    ]) * ureg.joule / ureg.mol,

    reaction_entropies=np.array([
        -80.0,   # dS1: A -> B
        -100.0,  # dS2: B -> C
        -150.0   # dS3: 2 A -> D
    ]) * ureg.joule / (ureg.mol * ureg.kelvin),
```

```
    reversible=np.array([
        True,
        True,
        True
    ])
)
```

The order of the species and reactions must remain consistent across all parameters. The rows of the stoichiometric matrix correspond to the species defined in `species`, while its columns correspond to the individual reactions. Consequently, the kinetic and thermodynamic parameter arrays contain one entry for each reaction.

In this example, all three reactions are reversible. The arrays containing the pre-exponential factors, activation energies, reaction enthalpies, and reaction entropies therefore describe reactions 1–3 in the same order as the columns of the stoichiometric matrix.

3.2.2 Reactor Models

The reactor model defines how the chemical source terms generated by the reaction mechanism are translated into changes of the reactor state. Different reactor models can therefore be combined with the same chemical mechanism.

In the following example, three reactor configurations are defined:

1. an ideal batch reactor,
2. an adiabatic plug-flow reactor with constant volumetric flow, and
3. a plug-flow reactor with external heat transfer.

```
reactor_list = [

    vr.BatchReactor(),

    vr.PlugFlowReactor(
        flow_model=vr.ConstantVolumetricFlow(
            1.5 * ureg.liter / ureg.minute
        )
    ),

    vr.PlugFlowReactor(
        flow_model=vr.ConstantVolumetricFlow(
            1.5 * ureg.liter / ureg.minute
```

```
        ),

        jacket_temperature=300 * ureg.kelvin,

        overall_heat_transfer_coefficient=(
            300
            * ureg.watt
            / (
                ureg.meter**2
                * ureg.kelvin
            )
        ),

        heat_transfer_area_per_volume=(
            200 / ureg.meter
        )
    )
]
```

The `BatchReactor` represents a closed reactor system in which the state evolves with time. No flow model is required because no material enters or leaves the reactor.

For a plug-flow reactor, the species state is represented by molar flow rates. The local concentrations therefore depend on the volumetric flow rate. In this example, the flow behavior is described by

$$\dot{V} = 1.5\ \mathrm{L\,min^{-1}}, \tag{3.10}$$

using the `ConstantVolumetricFlow` model.

The second plug-flow reactor is adiabatic with respect to external heat transfer, whereas the third configuration introduces heat exchange with an external jacket. The heat-transfer contribution is characterized by the jacket temperature

$$T_{\mathrm{jacket}} = 300\ \mathrm{K}, \tag{3.11}$$

the overall heat-transfer coefficient

$$U = 300\ \mathrm{W\,m^{-2}\,K^{-1}}, \tag{3.12}$$

and the heat-transfer area per reactor volume

$$a_V = 200\ \mathrm{m^{-1}}. \tag{3.13}$$

Together, U and a_V determine the strength of heat exchange between the reacting fluid and the external jacket.

3.2.3 Physical Property Model

Thermophysical properties required by the reactor energy balance are provided independently through a `PropertyModel`. This allows the same reactor configuration to be evaluated using different assumptions for the physical properties of the reacting mixture.

For the present example, constant density and heat capacity are assumed:

```
constant_density_heat_capacity = vr.PropertyModel.constant(
    density=1000.0 * ureg.kg / ureg.m**3,

    heat_capacity=(
        4180.0
        * ureg.joule
        / (
            ureg.kg
            * ureg.kelvin
        )
    )
)
```

The model therefore assumes

$$\rho = 1000 \text{ kg m}^{-3} \tag{3.14}$$

and

$$c_p = 4180 \text{ J kg}^{-1} \text{ K}^{-1}. \tag{3.15}$$

These values approximately correspond to a water-like liquid and are used here as a simple constant-property model.

Separating the physical property model from the reactor model makes it possible to exchange property descriptions independently of the reaction mechanism or reactor configuration. This is particularly useful for systematic parameter studies and the generation of structured simulation datasets.

3.2.4 Simulation and Initial Conditions

Once the reaction mechanism, reactor models, and physical properties have been defined, they are combined in a `Simulation` object. The initial state and integration interval depend on the selected reactor model.

For the three reactor configurations introduced above, the integration limits are defined as

```
xi_list = [300, 0.05, 0.05]
```

The corresponding initial states are

```
initial_states = [
    np.array([1, 0, 0, 0, 300, 0]),      # Batch
    np.array([2.5e-5, 0, 0, 0, 300, 0]), # PFR 1
    np.array([2.5e-5, 0, 0, 0, 300, 0]), # PFR 2
]
```

For the batch reactor, the state vector is

$$\mathbf{y}_{0,\mathrm{Batch}} = \begin{pmatrix} \mathbf{c}_0 \\ T_0 \\ p_0 \end{pmatrix}, \tag{3.16}$$

where the species entries represent initial concentrations. In the present example,

$$\mathbf{c}_0 = \begin{pmatrix} 1 & 0 & 0 & 0 \end{pmatrix}^{\mathrm{T}} \mathrm{mol\,m^{-3}}, \tag{3.17}$$

with an initial temperature of $T_0 = 300$ K.

For the plug-flow reactors, the state vector instead contains molar flow rates,

$$\mathbf{y}_{0,\mathrm{PFR}} = \begin{pmatrix} \mathbf{F}_0 \\ T_0 \\ p_0 \end{pmatrix}, \tag{3.18}$$

with

$$\mathbf{F}_0 = \begin{pmatrix} 2.5 \times 10^{-5} & 0 & 0 & 0 \end{pmatrix}^{\mathrm{T}} \mathrm{mol\,s^{-1}}. \tag{3.19}$$

The independent coordinate ξ is interpreted according to the reactor model. For the batch reactor,

$$\xi = t, \tag{3.20}$$

and the simulation is performed over

$$0 \leq t \leq 300 \text{ s}. \tag{3.21}$$

For the plug-flow reactors,

$$\xi = V, \tag{3.22}$$

and the reactor state is integrated over

$$0 \leq V \leq 0.05 \text{ m}^3. \tag{3.23}$$

Running the Simulations

The simulations are performed iteratively using the same reaction mechanism and physical property model while exchanging the reactor configuration:

```
results = []

for i in range(3):

    vdv_sim = vr.Simulation(
        mechanism=van_de_vusse,
        reactor=reactor_list[i],
        properties=constant_density_heat_capacity
    )

    vdv_sim_result = vdv_sim.solve(
        initial_state=initial_states[i],
        xi_span=(0, xi_list[i])
    )

    results.append(vdv_sim_result)
```

Each `Simulation` combines three independent model components:

$$\boxed{\texttt{Simulation} = \texttt{Mechanism} + \texttt{Reactor} + \texttt{PropertyModel}} \tag{3.24}$$

The `solve()` method integrates the reactor state from the specified initial condition over the interval `xi_span`. The resulting `SimulationResult` is appended to `results`, allowing the different reactor configurations to be analyzed and compared within the same workflow.

This modular initialization makes it possible to systematically vary reactor models, operating conditions, or physical property models while keeping the chemical mechanism unchanged.

3.3 Simulation Results

The `solve()` method performs the numerical integration and returns a `SimulationResult` object,

$$\texttt{Simulation} \xrightarrow{\texttt{solve()}} \texttt{SimulationResult}. \tag{3.25}$$

A `SimulationResult` represents the complete output of a single simulation. It stores the values of the independent coordinate ξ together with the corresponding reactor states $\mathbf{y}(\xi)$.

The result of a simulation can therefore be obtained as

```
result = simulation.solve(
    initial_state=initial_state,
    xi_span=(0, xi_end)
)
```

Conceptually, the numerical solution consists of

$$\{\xi_i,\, \mathbf{y}(\xi_i)\}_{i=1}^{N}\,, \tag{3.26}$$

where N is the number of evaluated points along the simulation coordinate.

For a batch reactor, the stored solution corresponds to

$$\{t_i,\, \mathbf{c}(t_i),\, T(t_i),\, p(t_i)\}\,, \tag{3.27}$$

whereas for a plug-flow reactor it corresponds to

$$\{V_i,\, \mathbf{F}(V_i),\, T(V_i),\, p(V_i)\}\,. \tag{3.28}$$

Separating the numerical result from the `Simulation` object allows simulation data to be analyzed, plotted, compared, and stored independently of the numerical integration itself.

3.4 Serialization

Simulation results can be serialized to HDF5 for persistent storage and subsequent analysis. The file contains the numerical solution together with metadata required to interpret the stored reactor state.

A `SimulationResult` can be stored using

```
result.save_hdf5(
    "simulation.h5"
)
```

The resulting HDF5 file has the following structure:

```
simulation.h5
|
+-- coordinates
+-- states
+-- species
|
+-- attributes
    +-- file_format
    +-- schema_version
    +-- coordinate_name
    +-- species_state_name
    +-- success
    +-- message
```

The `coordinates` dataset contains the independent simulation coordinate ξ, while `states` contains the corresponding reactor states $\mathbf{y}(\xi)$. The `species` dataset stores the associated species names.

The HDF5 attributes provide additional information required to interpret the result. In particular, `coordinate_name` identifies the independent coordinate, such as time or

reactor volume, and `species_state_name` identifies the representation of the species states, such as concentrations or molar flow rates.

A stored result can be reconstructed using

```
result = vr.SimulationResult.load_hdf5(
    "simulation.h5"
)
```